

UNIT IV

PACKAGES, EXCEPTION HANDLING AND I/O STREAMS

PACKAGES:

A package is a namespace that organizes related classes and interfaces. Conceptually, it's similar to folders in a file system. Packages in Java are a fundamental organizational feature that help manage large codebases, prevent naming conflicts, and control access to classes and interfaces.

Types of Packages:

- **Built-in Packages:**

These packages are provided by the Java API (Application Programming Interface). A vast library of pre-written classes and interfaces is included, ready to be used in Java programs.

Some of the packages are

- `java.lang` (automatically imported)
- `java.util` (collections framework)
- `java.io` (input/output)
- `java.net` (networking)
- `java.sql` (database access)

- **User-defined Packages:**

These packages are created by developers to organize their own classes and interfaces based on functionality or module. To create a user-defined package, the ***package*** keyword followed by the package name is used at the beginning of a Java source file.

```
package packname;
```

Simple User-defined package creation and import

- Create Java file with package declaration

```
// File: Calculator.java
package mathoperations;
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
    public int subtract(int a, int b) {
        return a - b;
    }
}
```

- Compile the package

```
javac -d . Calculator.java
```

This will create a ***mathoperations*** directory containing the compiled *.class* files.

The `-d` is a switch that tells the compiler where to put the class file i.e. it represents destination. The `.` (dot) represents the current folder.

- Create a test class outside the package by importing *Calculator* class from *mathoperations* package.

```
// File: TestCalculator.java
import mathoperations.Calculator;
public class TestCalculator {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println("5 + 3 = " + calc.add(5, 3));
        System.out.println("5 * 3 = " + calc.subtract(5, 3));
    }
}
```

- Compile and run the test

```
javac TestCalculator.java
java TestCalculator
```

- Output

```
5 + 3 = 8
5 - 3 = 2
```

Importing Packages in Java:

In Java, packages are imported to access classes and interfaces defined in them. Here are the common ways to import packages:

- **Import a Specific Class**

```
import java.util.Scanner;
```

Only the `Scanner` class from the `java.util` package is imported.

- **Import All Classes from a Package (Wildcard Import)**

```
import java.util.*;
```

Imports all classes and interfaces from the `java.util` package. Does not include sub-packages.

- **Static Import (for Static Members)**

```
import static java.lang.Math.*;
```

Allows direct access to static methods like `sqrt()`, `pow()` without `Math.` prefix.

- **Fully Qualified Name (No Import)**

```
public class Test {  
    public static void main(String[] args) {  
        java.util.Scanner sc = new java.util.Scanner(System.in);  
    }  
}
```

No import statement; class is accessed using full package path.

Member Access:

- In Java, package member access refers to how classes and class members (fields, methods, constructors) within a package can be accessed from other classes either within the same package or from different packages. Access is controlled using access modifiers.
- Member Access Rules
 - **Private members:** Accessible only within the declared class.
 - **Default (no modifier):** Accessible only within the same package.
 - **Protected members:** Accessible in the same package and in subclasses (even if in a different package).
 - **Public members:** Accessible from anywhere.

Modifier	Same Class	Same Package	Subclass (other package)	Other Package
private	Yes	No	No	No
(default)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

- **Example:**

Assume following is the directory structure.

```
src/  
├── package1/  
│   └── ClassA.java  
└── package2/  
    └── ClassB.java
```

ClassA.java (Defines members with different access modifiers)

```
// File: package1/ClassA.java  
package package1;  
  
public class ClassA {  
    private int privateVar = 1;  
    int defaultVar = 2;           // default access  
    protected int protectedVar = 3;  
    public int publicVar = 4;  
  
    public void display() {  
        System.out.println("Inside ClassA:");  
        System.out.println("privateVar: " + privateVar);  
        System.out.println("defaultVar: " + defaultVar);  
        System.out.println("protectedVar: " + protectedVar);  
        System.out.println("publicVar: " + publicVar);  
    }  
}
```

ClassB.java (Accessing ClassA's members from a different package)

```
// File: package2/ClassB.java  
package package2;  
  
import package1.ClassA;  
public class ClassB extends ClassA {  
    public static void main(String[] args) {  
        ClassA obj = new ClassA();  
        obj.display(); // Works: public method  
    }  
}
```

```
        System.out.println("\nAccessing from different package:");

        // Not accessible
        // System.out.println("privateVar: " + obj.privateVar);
        // System.out.println("defaultVar: " + obj.defaultVar);

        //Not accessible via object
        // System.out.println("protectedVar: " + obj.protectedVar);

        // Accessible
        System.out.println("publicVar: " + obj.publicVar);

        // Accessing protected member via inheritance
        ClassB subObj = new ClassB();
        System.out.println("protectedVar (via subclass): " +
                           subObj.protectedVar); // Accessible
    }
}
```

EXCEPTION HANDLING:

- An Exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. It typically occurs due to logical errors or system-generated errors such as: Division by zero, Accessing a null object, Invalid array index.
- Exception Handling in Java is a mechanism to handle runtime errors, allowing the normal flow of the program to continue. It involves detecting, handling, and resolving exceptions that occur during program execution, such as divide-by-zero, invalid input, or file-not-found errors.
- It prevents program termination and provides a way to gracefully handle errors.
- **Types of Exceptions:**

Exception Type	Description	Examples
Checked Exception	Exceptions that are checked by the compiler at compile-time. Must handle or declare.	IOException, SQLException
Unchecked Exception	Exceptions that are not checked by the compiler at compile-time, occur during runtime.	NullPointerException, ArithmeticException
Error	Serious problems caused by the JVM (Java Virtual Machine) that applications should not try to catch or handle.	OutOfMemoryError, StackOverflowError

Exception Hierarchy:

```

java.lang.Object
├── java.lang.Throwable
│   ├── java.lang.Exception
│   │   ├── java.io.IOException
│   │   ├── java.sql.SQLException
│   │   ├── java.lang.RuntimeException
│   │   │   ├── ArithmeticException
│   │   │   ├── NullPointerException
│   │   │   ├── ArrayIndexOutOfBoundsException
│   │   │   └── ...
│   │   └── Other Checked Exceptions
│   └── java.lang.Error
│       ├── OutOfMemoryError
│       ├── StackOverflowError
│       ├── VirtualMachineError
│       └── ...

```

Keywords used in Exception Handling:

Keyword	Description	Syntax
try	Contains code that might throw an exception.	try { // Code that may throw an exception }
catch	Handles the exception thrown by the try block.	catch (ExceptionType e) { // Code to handle the exception }
finally	Executes code after try-catch, regardless of the exception (optional).	finally { // Code that will always execute //(cleanup, closing files, etc.) }
throw	Used to explicitly throw an exception.	throw new ExceptionType("Error message");
throws	Declares exceptions a method might throw (used in method signature).	returnType methodName() throws ExceptionType { /* Method code that might throw Exceptions */ }

Example-1: try-catch-finally with single exception.

```
class ExceptionHandlingDemo {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // Throws ArithmeticException
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: Cannot divide by zero.");
        } finally {
            System.out.println("Finally block executed (cleanup code).");
        }
    }
}
```

Example-2: try-catch-finally with multiple exceptions.

```
class MultiExceptionExample {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            // Throws ArrayIndexOutOfBoundsException
            System.out.println(numbers[5]);
            int result = 10 / 0; // Throws ArithmeticException
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception caught: Invalid array index.");
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: Cannot divide by zero.");
        } catch (Exception e) {
            System.out.println("Some thing went wrong!!");
        }
        finally {
            System.out.println("Finally block executed (cleanup code).");
        }
    }
}
```

Note: When a program contains multiple exceptions, they should be handled in a specific order to ensure that the most specific exceptions are caught before the general exceptions.

Nested try-catch blocks:

- Nested try-catch means placing a try-catch block inside another try or catch block. This allows handling different exceptions at different levels of the code.

Example:

```
class NestedTryCatchExample {  
    public static void main(String[] args) {  
        try {  
            int[] numbers = {1, 2, 3};  
            try {  
                // Throws ArrayIndexOutOfBoundsException  
                System.out.println(numbers[10]);  
            }  
            catch (ArrayIndexOutOfBoundsException e) {  
                System.out.println("Inner catch: Invalid array index!");  
            }  
            int result = 10 / 0; // Throws ArithmeticException  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Outer catch: Cannot divide by zero!");  
        }  
        System.out.println("Program continues...");  
    }  
}
```

User-defined Exceptions:

- User-defined exceptions are custom exceptions created by extending the Exception class (or its subclass). These are useful to throw and handle exceptions that are specific to your application logic.
- Create a User-Defined Exception
 1. Extend the Exception class

```
class MyException extends Exception {  
    public MyException(String message) {  
        super(message);  
    }  
}
```


2. Throw the Exception

```
public class Test {  
    static void checkAge(int age) throws MyException {  
        if (age < 18) {  
            throw new MyException("Age is less than 18 - Not Eligible");  
        } else {  
            System.out.println("Eligible");  
        }  
    }  
  
    public static void main(String[] args) {  
        try {  
            checkAge(16);  
        } catch (MyException e) {  
            System.out.println("Caught exception: " + e.getMessage());  
        }  
    }  
}
```

Output:

```
Caught exception: Age is less than 18 - Not Eligible
```

- *Checked* exceptions are created by extending the *Exception* class and must be declared with the *throws* keyword.
- *Unchecked* exceptions are created by extending the *RuntimeException* class and may be declared, but it is *not mandatory* to use the *throws* keyword.

throw vs throws:

throw is a Java statement used to explicitly throw an exception during program execution. It is used inside a method or block to manually cause an exception. It must be followed by an exception object.

Syntax:

```
throw new ExceptionType("Error Message");
```

Example:

```
throw new ArithmeticException("Cannot divide by zero");
```

throws is a Java keyword used to declare exceptions that a method might throw to the caller. It is used in the method signature. It tells the caller to handle the exception.

Syntax:

```
returnType methodName(parameters) throws ExceptionType1, ExceptionType2 {
    // method body
}
```

Example:

```
void readFile() throws IOException {
    FileReader fr = new FileReader("file.txt");
}
```

throw	throws
1. It is a Java Statement	1. It is a Java Keyword
2. Used to manually throw an exception	2. Used to declare exceptions that a method may throw
3. Use inside a method or block	3. Use in method signature
4. Actually, throws an exception	4. Informs the caller about possible exceptions
5. Can throw only one exception at a time	5. Can declare multiple exceptions , separated by commas
6. Used for both types	6. Used mostly for checked exceptions

I/O STREAMS (FILE HANDLING):

- Streams can be classified into types
 - Byte streams
 - Text streams (or) Character streams
- Byte streams** are used to handle any characters (text), images, audio, and video files. It represents data in the form of individual bytes.
- For example, to store an image file (.gif or \jpg), we should go for a byte stream.
- Byte stream classes for reading and writing the data

```
FileInputStream
FileOutputStream
BufferedInputStream
BufferedOutputStream
```

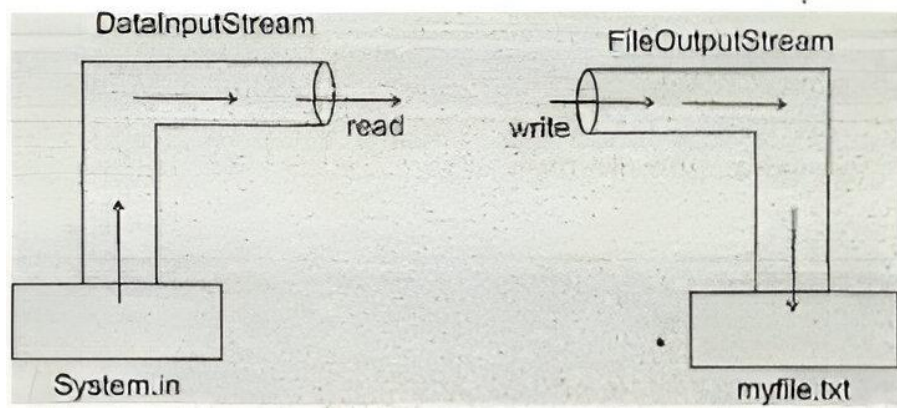
- Character or text streams** can always store and retrieve data in the form of characters or text only.
- It means character or text streams are more suitable for handling text files like the ones we create in Notepad. They are not suitable to handle the images, audio, or video files.

- Character streams classes for reading and writing data

```
FileReader  
FileWriter  
BufferedReader  
BufferedWriter
```

Creating a file using FileOutputStream:

- FileOutputStream* class belongs to byte stream and stores the data in the form of individual bytes. *DataInputStream* is used to read data from keyboard.
- It can be used to create text files. We know that a file represents storage of data on a second storage media like a hard disk or CD.



- Example:

```
import java.io.*; class CreateFile  
{  
    public static void main(String args[])throws IOException  
    {  
        DataInputStream dis=new DataInputStream(System.in);  
        FileOutputStream fout=new FileOutputStream("myfile.txt");  
        System.out.println("enter text(give @ at end)");  
        char ch;  
        while((ch=(char)dis.read())!='@'){  
            fout.write(ch); fout.close();  
        }  
    }  
}
```

Compilation: javac CreateFile.java

Execution: java CreateFile
enter text(give @ at end)

```
hello this is file handling concept.  
@
```

To see file content: *myfile.txt*

Appending text to already existing file:

- For appending text to already existing file we have use the following form

```
FileOutputStream fout=new FileOutputStream("myfile.txt",true);
```

- Example:

```
import java.io.*;  
class CreateFile1  
{  
    public static void main(String args[])throws IOException  
    {  
        DataInputStream dis=new DataInputStream(System.in);  
        FileOutputStream fout=new FileOutputStream("myfile.txt",true);  
        System.out.println("enter text(give @ at end)");  
        char ch;  
        while((ch=(char)dis.read())!='@'){  
            fout.write(ch);  
        }  
        fout.close();  
    }  
}
```

Compilation: javac CreateFile1.java

Execution: java CreateFile1
enter text(give @ at end)
hello this is appended text
@

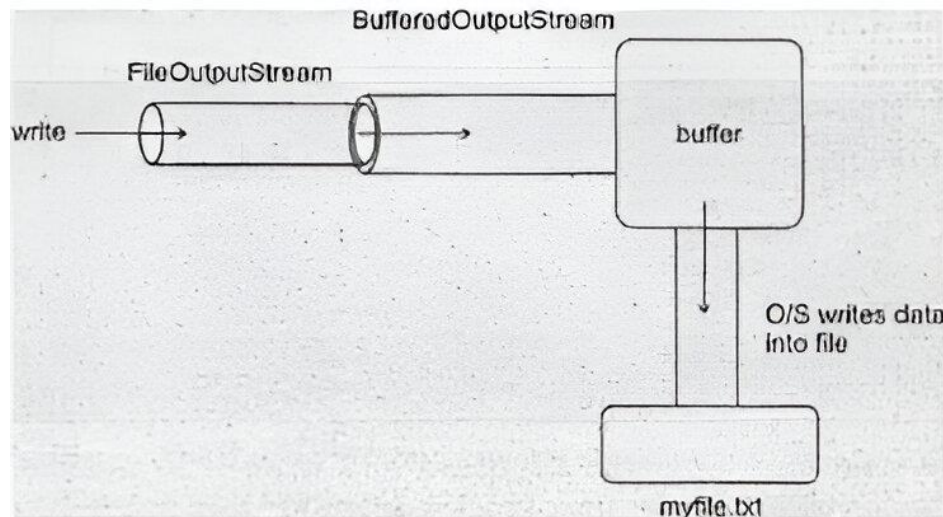
To see file content: *myfile.txt*

Creating a file using BufferedOutputStream:

- While creating a file using *BufferedOutputStream* class the efficiency of the program is improved. *BufferedOutputStream* takes argument as *FileOutputStream* as reference for creating a Buffer. The default size of Buffer is 512 bytes.
- The general form of *BufferedOutputStream* class is

```
BufferedOutputStream bout=new BufferedOutputStream(fout,1024);
```

- In the above form “fout” means reference of *FileOutputStream* class and 1024 is bytes.



- **Example**

```
import java.io.*;
class CreateFile2
{
    public static void main(String args[]) throws IOException
    {
        DataInputStream dis=new DataInputStream(System.in);
        FileOutputStream fout=new FileOutputStream("myfile1.txt");
        BufferedOutputStream bout=new BufferedOutputStream(fout,1024);
        System.out.println("enter text(give @ at end)");
        char ch;
        while ((ch=(char)dis.read())!='@')
            bout.write(ch);
        bout.close();
    }
}
```

Compilation: javac CreateFile2.java

Execution: java CreateFile2
 enter text(give @ at end)
 hello this is BufferedOutputStream class. @

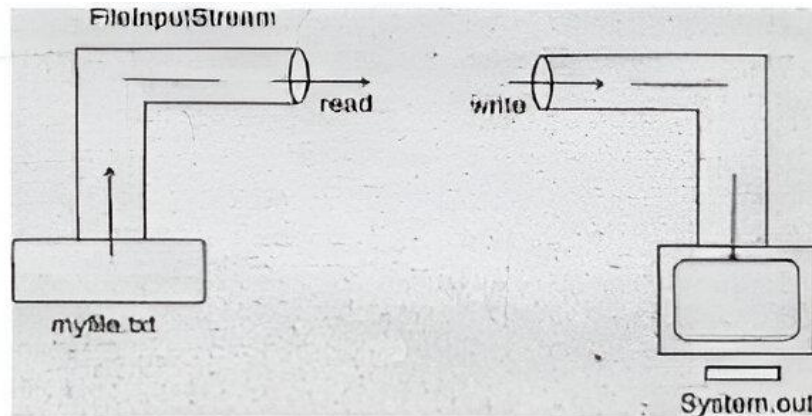
To see file content: type myfile1.txt

Ouput: hello this is BufferedOutputStream class.

Reading data from a file using FileInputStream:

- FileInputStream is useful to read data from a file in the form of sequence of bytes.
- It is possible to read data from a text file using FileInputStream.

```
FileInputStream fin=new FileInputStream("myfile.txt")
```

**Program:**

```
import java.io.*;
class ReadFile
{
    public static void main(String args[]) throws IOException
    {
        FileInputStream fin=new FileInputStream("myfile.txt");
        System.out.println("File contents:");
        int ch;
        while((ch=fin.read())!=-1)
            System.out.print((char)ch);
        fin.close();
    }
}
```

Compilation: javac CreateFile1.java**Execution:** java CreateFile1**Output:**File contents:
hello this is file handling concept.hello this is appended text.**Creating a file using FileWriter:**

- FileWriter is useful to create a file by writing characters into it.
- The following program shows how to create a text file using FileWriter.

Program:

```
import java.io.*;
class CreatingFile
{
    public static void main(String args[]) throws IOException
    {
        DataInputStream dis=new DataInputStream(System.in);
        FileWriter fw=new FileWriter("myfile2.txt");
        System.out.println("enter text(give @ at end)");
        char ch;
        while((ch=(char)dis.read())!='@')
            fw.write(ch);
        fw.close();
    }
}
```

Compilation: javac CreatingFile.java

Execution: java CreatingFile

enter text(give @ at end)

hello we are CSE students @

To see file content: myfile2.txt

hello we are CSE students

Reading data from a file using FileReader:

- FileReader is useful to read data in the form of characters from a 'text' file.

- **Program:**

```
import java.io.*;
class ReadingFile
{
    public static void main(String args[]) throws IOException
    {
        FileReader fr=new FileReader("myfile2.txt");
        System.out.println("File contents:");
        int ch;
        while((ch=fr.read())!=-1)
            System.out.print((char)ch);
        fr.close();
    }
}
```

Compilation: javac ReadingFile.java

Execution: java ReadingFile

Output:

hello we are CSE students

File class:

- File class of java.io package provides some methods to know the properties of a file or a directory.
- Creating the File object by passing the filename or directory name to it.

```
File obj=new File("filename");
File obj=new File("directory name");
File obj=new File("path","filename");
File obj=new File("path","directoryname");
```

- We can create text files using File class by replacing “FileOutputStream” and “FileWriter” class with “File” class.

Creating file using File class:

```
import java.io.*;
class CreateFile2
{
    public static void main(String args[])throws IOException
    {
        DataInputStream dis=new DataInputStream(System.in);
        File f=new File("myfile1.txt");
        System.out.println("enter text(give @ at end)");
        char ch;
        while((ch=(char)dis.read())!='@')
            f.write(ch);
        f.close();
    }
}
```

Creating directories using File class:

```
import java.io.File;
class CreateDirectoryExample
{
    public static void main(String[] args)
    {
        // creating a directory in current working directory
        File file = new File("mydir");
        // exists() returns Boolean value true or false
        if (!file.exists())
        {
            // mkdir() returns Boolean value true or false
            if (file.mkdir())
                System.out.println("Directory is created!");
            else
                System.out.println("Failed to create directory!");
        }
        // creating multiple directories in specified path
        File files = new File("C:\\Directory2\\Sub2\\Sub-Sub2");
        if (!files.exists())
        {
            if (files.mkdirs())
                System.out.println("Multiple directories are created!");
            else
                System.out.println("Failed to create multi directories!");
        }
    }
}
```

Counting number of lines, words and characters in a text file:

```
import java.io.*; class
WordCountInFile
{
    public static void main(String[] args)throws IOException
    {
        BufferedReader reader = null; int
        charCount = 0;
        int wordCount = 0;
        int lineCount = 0;
        try
        {
            FileReader fr=new FileReader("myfile.txt");
            reader = new BufferedReader(fr);
            String currentLine = reader.readLine(); while
            (currentLine != null)
            {
                lineCount++;
                String[] words = currentLine.split(" ");
                wordCount = wordCount + words.length; for
                (String word : words)
                {
                    charCount = charCount + word.length();
                }
                currentLine = reader.readLine();
            }
            System.out.println("Chars In a File: "+charCount);
            System.out.println("Words In a File: "+wordCount);
            System.out.println("Lines In a File: "+lineCount);
        }
        catch (IOException e)
        {
            System.out.println(e);
        }
        finally
        {
            reader.close();
        }
    }
}
```

Compilation: javac WordCountInFile.java

Execution: java WordCountInFile

Output:

//Based on your file data displays output.

Reading data from console using BufferedReader:

```
import java.io.*;
class ReadFromConsole
{
    public static void main(String args[]) throws Exception
    {
        DataInputStream dis=new DataInputStream(System.in);
        BufferedReader br=new BufferedReader(dis);
        System.out.println("Enter your name");
        String name=br.readLine();
        System.out.println("Welcome "+name);
    }
}
```

Compilation: javac ReadFromConsole.java

Execution: java ReadFromConsole

Output:

Enter your name Sachin

Tendulkar

Welcome Sachin Tendulkar
